

Muster

A fleet inventory, compliance, and security-posture platform, built from scratch in Go — with a live walkthrough, the reasoning behind the design, and answers to the questions a technical audience is likely to ask.

"What does every host in my fleet actually look like, right now, and can I prove it to an auditor?"

~25,500

LINES OF
GO

36

COMMITTS

25

SCREENSHOTS

MIT

LICENSE

● What's in this script

This is a working script for a live, technical deep-dive: enough narration to carry the demo, enough architecture to satisfy an engineer in the room, and a prepared Q&A for the questions that reliably come up.

1. Why I built this	The motivation, and what it's meant to prove
2. Thirty-second pitch	The opening line, verbatim
3. Architecture, in one breath	How data gets in, and what happens to it
4. The demo walkthrough	Nine steps, screen by screen, with talking points
5. What's real vs. what's a fallback	The honesty section — say this before anyone asks
6. Anticipated Q&A	Nineteen questions a technical audience is likely to ask
7. If something breaks	A recovery line for each demo step

1 Why I built this

Say this early, in your own words — it's the difference between "here's a project" and "here's why you should care."

I spent years around **InfraAtlas**, a 20-year-old Perl inventory system that a team depended on but nobody wanted to touch. It did real work — it just did it in a codebase that had grown organically for two decades. Modernizing it taught me exactly which parts of that domain are actually hard: matching a vulnerability to the right installed package, keeping a compliance framework's rules from drifting out of sync with what the checks actually test, forwarding an audit trail somewhere durable without losing events when the downstream system hiccups.

Muster is **not a port** of that codebase. It's a clean-room rebuild of the same problem space — fleet hardware, software, and configuration inventory, plus the compliance and security-posture layer on top — designed and built from nothing, so every decision in it is one I made on purpose rather than one I inherited. I picked Go deliberately: the core of this system is a concurrent ingest daemon and a text-parsing pipeline, which is exactly where Go is strong, and it's the same language behind Prometheus, the OpenTelemetry Collector, Terraform, and Kubernetes — the tools most infra and security teams already run.

SAY IT LIKE THIS

"I built Muster to prove two things to myself: that I could take a real, messy infrastructure problem and design a clean solution for it from a blank page, and that I could hold myself to production discipline the whole way — compiled, vetted, tested, and exercised against a running server before anything got committed. It's a portfolio piece, but I built it like I'd be the one getting paged for it."

One design decision is worth calling out on its own, because it came from a real moment during development, not from a spec: when I first pointed Ask Muster — the natural-language query feature — at a real model, it needed an Anthropic API key, which meant fleet data (hostnames, IPs, installed software, CVEs) leaving the network with every question. That's a legitimate objection a security buyer would raise. So I made the model backend pluggable, the same way SIEM forwarding already was, so the same feature can run against a model hosted entirely inside the operator's own network. That single change turns Ask Muster from a review risk into a differentiator — **it's the best "why" story in the whole demo**, and it's covered step-by-step in the walkthrough.

2 Thirty-second pitch

Open with this, verbatim or close to it, before touching the keyboard:

OPENING LINE

"Muster answers one question for a fleet of any size: what does every host actually look like right now, and can you prove it? It pulls hardware, software, and configuration facts from real agents, scores each

host against compliance frameworks like CIS, HIPAA, and NIST 800-53, correlates installed packages against known CVEs, and puts all of it — including an audit trail of every question anyone asks it — behind one dashboard. It's about twenty-five thousand lines of Go, dependency-free vanilla JS on the frontend, and everything you're about to see is running against a seeded demo fleet on a box on my network, not a mockup."

3 Architecture, in one breath

Have this ready if anyone asks "how does data get in," before you reach the architecture diagram on screen:

An **agent** (Linux, Windows, macOS, ChromeOS, Android, iOS, or an air-gapped tar.gz drop) collects host facts and ships them over a token-authenticated TCP protocol to the **ingest daemon**, which extracts and hands them to a **cook/parse pipeline**. That pipeline normalizes everything into the **Store** interface — Postgres in production, in-memory for tests — optionally wrapped so every write also forwards to a SIEM. A background **evaluator** loop continuously ties policy, compliance posture, vulnerability correlation, score history, and alerting together. Everything above that is a REST API and a dependency-free web dashboard reading the same store.

Cloud scanners (AWS/Azure/GCP), the network discovery scanner, and third-party vulnerability scanners (Nessus, Qualys, generic CSV) all report into the same pipeline through their own entry points, so nothing downstream — compliance checks, risk scoring, Ask Muster's context, CSV exports — has to know or care where a fact originated.

3

STORE BACKENDS VERIFIED
(POSTGRES, MEMSTORE,
TESTS)

25+

REST ENDPOINTS ADDED
THIS DEVELOPMENT ROUND

62 / 82

ENTITIES / RELATIONSHIPS
IN THE DEMO FLEET'S MAP

4 The demo walkthrough

Nine steps, in the order they build on each other. Each has a talking point and the one line to say if the audience only remembers one thing from that screen.

1

FLEET TAB

The fleet dashboard

Open on the Fleet tab. Point at the 30-day trend chart and the per-host risk view before clicking anything.

SAY

This used to be Michael's own complaint about the old system — "no dashboards, no trends." Every host gets a compliance score, a posture score, and a blended risk score from vulnerability severity, internet exposure, criticality tags, and staleness, and it's all tracked over time, not just as a snapshot.

2

HOST DETAIL PAGE

Drill into one host

Click into a host with known vulnerabilities (e.g. `db01.prod`). Show installed software, the vulnerability list with CVE detail, and the host's own history sparkline.

SAY

Every fact on this page — packages, open ports, certificates, browser extensions — came from a real agent collection, and every vulnerability match ties a specific installed package version to a specific CVE, not a fuzzy keyword hit.

3

COMPLIANCE TAB

Switch frameworks live

Show the framework selector: CIS-style baseline, HIPAA Security Rule, NIST 800-53. Switch between them with `?framework=` and watch the pass/fail counts change.

SAY

The framework is a request parameter, not a rebuild — that's the proof it's actually pluggable and not just one hard-coded checklist wearing three names.

4

COMPLIANCE TAB → SCANNER IMPORT

Import a third-party scan

Upload a sample Nessus, Qualys, or generic CSV export. Show findings matched to hosts by FQDN, short name, or IP, and flowing into the same compliance and risk numbers as Muster's own matching — tagged by source.

SAY

This isn't a second, disconnected vulnerability list. Once it's imported, it's just more evidence feeding the same scores, clearly labeled so the two sources never get blurred together.

5

ENTITY MAP TAB

The entity relationship map

Open the Entity map. Click a CVE node to pivot and show every host it touches; click a rule to show what it actually governs.

SAY

Every one of these relationships already existed somewhere in the system — this is the first place they're shown together. The hard part wasn't drawing it, it was noise: thousands of installed packages become about twenty nodes, because a package only earns a node if it's something an operator would act on.

6

ASK MUSTER PANEL — THE CENTERPIECE

Ask Muster, running on a self-hosted model

Ask a real question — "Any shadow IT I should know about?" or "What prod hosts have vulnerabilities?" — and let it answer from the running Qwen3 model on local hardware.

SAY

This is a natural-language interface over your real fleet data, and every question and answer is written to the audit log — it's governed, not a generic chatbot bolted on. And the model answering right now isn't a third-party API: it's running on hardware I control, on my own network, which matters because every question ships real hostnames, IPs, software, and CVEs as context. That's not a feature I added for a demo — I built it after hitting the "no Anthropic key configured" error live and asking myself whether the answer really needed to leave the building.

TIMING NOTE

The self-hosted model takes roughly 35–40 seconds to answer (prefill on ~1,600 tokens of fleet context, then generation on CPU-only hardware). Ask the question, then keep talking through step 5's design decisions or the architecture recap while it finishes — don't stand in silence.

7

REPORTS & EXPORTS

Executive summary and CSV exports

Open the print-ready executive summary (Ask Muster-authored, with a non-AI template fallback), and show a CSV export.

SAY

Same answer to "no reports" — this opens as a print-ready page today, and it's written by the same governed AI path, with a deterministic template fallback if no model is configured at all.

8

SETTINGS TAB

Live reconfiguration, and the demo simulator

In Settings, show the Ask Muster backend dropdown (Anthropic / Hugging Face / self-hosted) and the SIEM forwarding backend (Splunk, Sumo Logic, LogRhythm) — both switchable with no restart. Then fire a synthetic event from the demo simulator and show it land in the audit log tagged `(simulated)`.

SAY

Nothing here needs a redeploy to change — that's what "pluggable" actually means in this codebase, and it's the same seam for both the AI backend and SIEM forwarding.

9

AUDIT LOG / STATUS PAGE

Close on governance

Show the audit trail entry for the question just asked, and the public `/status` page.

SAY

Every AI answer, every settings change, every API key use is attributable and logged. That's the difference between "a chatbot that happens to see your data" and something a security team would actually sign off on.

5 What's real vs. what's a fallback

Say this proactively, before Q&A — it earns more credibility than it costs, and it heads off the "is this all smoke and mirrors" question before it's asked.

Verified for real

- `go build`, `go vet`, and `go test` clean after every commit, including the Postgres integration suite against a real Postgres 16 instance.
- Every dashboard route swept headlessly for console errors and failed cards — zero of both, across all 11+ tabs.
- The Linux agent collecting real browser extensions and certificates, air-gap import, live SIEM reconfiguration, notification retry/dead-lettering, group-scoped API key enforcement, and a real HavelBeenPwned lookup.
- The self-hosted Ask Muster path, driven end-to-end against a real Qwen3 model on real hardware — including catching and fixing a genuine bug (below).

Explicitly not verified (say this out loud)

- Windows and macOS agents, the three cloud-scanner integrations, and OAuth against a real identity provider — no such environment was available to test against; all are unit tested against mocked interfaces.
- Live Slack, Teams, Jira, and ServiceNow webhook delivery — clients are written from published API docs and unit tested against `httptest`, not fired at real tenants yet.
- A real Nessus or Qualys export — the sample CSVs were hand-built to each vendor's documented column layout.

A BUG WORTH TELLING, NOT HIDING

The first real question asked of a self-hosted reasoning model (Qwen3) came back as an empty answer with no explanation. Root cause: reasoning models write a hidden "thinking" scratchpad before their visible answer, charged against the same token budget — so a budget sized for a normal model got consumed entirely by thinking. This is exactly the kind of failure that only shows up when you run the real thing instead of a stub. It's fixed now — a safety floor on the token budget, and an error message that names the model and the actual cause instead of reading like a network fault — and there's a regression test pinned to the exact response shape that broke it.

6 Anticipated Q&A

Grouped by theme. Answer in your own words — these are the facts to hit, not a script to recite.

Design and motivation

Q. Why build this instead of just using an existing tool?

A. This is a portfolio project, not a product pitch — the point was to prove I could design and build the hard parts of a fleet inventory/compliance system myself: agent protocol, fact parsing, vulnerability correlation, pluggable backends. In a real org I'd absolutely evaluate osquery, Wazuh, or a CSPM platform first.

Q. Why Go instead of Python or Node?

A. The core is a concurrent ingest daemon plus a text-parsing pipeline — Go's strengths — and it's the same language behind Prometheus, the OTel Collector, Terraform, and Kubernetes, which signals directly to the infra/security roles I'm targeting.

Q. Is this related to InfraAtlas?

A. InfraAtlas is a 20-year-old Perl system I worked on modernizing — it's where the domain knowledge came from. Muster is a clean-room rebuild, not a port: different protocol, different data model, different codebase, built from a blank page.

The AI feature

Q. Isn't sending fleet data to an LLM a security risk?

A. That's exactly the objection that led to the pluggable backend. Ask Muster can run against Anthropic, any Hugging Face model, or — the point of this demo — a model on hardware the operator controls, so fleet context never leaves the network. Both the policy drafter and the summary generator also have non-AI template fallbacks.

Q. How does one implementation reach three different providers?

A. Hugging Face's Inference Providers router, Ollama, LM Studio, vLLM, and text-generation-inference all speak the OpenAI chat-completions wire format. One HTTP client implementation reaches all of them; only the base URL changes.

Q. What hardware is the model running on, and how fast is it?

A. A CPU-only VM (Xeon E5-2697 v4, no GPU) running Qwen3-30B-A3B via Ollama — a mixture-of-experts model with about 3B active parameters per token, so it runs near 3B speed at near-30B quality. Measured: 24.2 tok/sec generation, 64.2 tok/sec prompt eval, ~35–40 seconds per answer including the ~1,600 tokens of fleet context sent with every question.

Q. What's the tradeoff versus a frontier model?

A. A small self-hosted model is measurably worse at open-ended generation — drafting a syntactically valid policy rule from plain English, specifically — which is exactly why that feature keeps a validation step and a non-AI fallback rather than trusting the model's output blindly.

Q. Could the model hallucinate a wrong answer about the fleet?

A. It answers from real context assembled from the Store, not from training-data recall, and every question and answer is logged to the audit trail for review — but yes, it's still a model, which is why nothing it drafts (a policy rule, a summary) ships without a deterministic validation or fallback path behind it.

Data and correctness

Q. How does vulnerability matching actually work?

A. Installed package name and version, parsed per-OS from real agent collection, matched against CVE data — not a keyword or fuzzy string match. Imported third-party scanner findings (Nessus/Qualys/CSV) flow through the same compliance checks and risk scoring, tagged by source so the two never get blurred.

Q. How do you know the numbers on screen are real and not hard-coded?

A. The whole UI is driven from a seeded demo fleet through the same Store interface production data would use — same API, same evaluator loop, same compliance engine. Nothing in the dashboard is templated with fake numbers.

Q. What's the entity map's node limit, and does it scale?

A. It's capped at 300 nodes server-side with a focus-and-depth pivot for larger graphs; the demo fleet produces 62 entities, so the cap itself hasn't been stress-tested against a larger fleet yet — that's an open item, not a hidden claim.

Production readiness

Q. Is this production-ready?

A. It's demo- and portfolio-ready, verified with real builds, real tests, and a real Postgres instance. It's explicitly not production-hardened: no SAML SSO yet, no multi-tenancy, and several integrations (live Slack/Teams/Jira/ServiceNow, cloud scanners, OAuth) are unit tested but not yet fired at real accounts.

Q. How is it deployed?

A. Directly with `go run`, via Docker Compose, a Kubernetes Helm chart, or standalone systemd — the running demo is a systemd service on an Ubuntu box, deployed from a Go binary with zero runtime dependencies beyond Postgres.

Q. What's the audit/compliance story for the audit log itself?

A. Every write path — settings changes, AI questions and answers, API key use, remediation approvals — goes through the same audit trail, and it can forward live to Splunk, Sumo Logic, or LogRhythm, switchable with no restart.

Q. What would you build next if this were a real product?

A. In priority order: SAML SSO and real multi-tenancy for an MSSP angle, live testing against real Slack/Jira/ServiceNow tenants and a real Nessus/Qualys export, and closing out the Windows/macOS/ChromeOS agent verification that's currently only unit tested.

Scope and scale

Q. How big a fleet could this handle?

A. Untested at scale — the demo fleet is 15 hosts. The architecture (Postgres-backed, stateless API, background evaluator loop) has no obvious ceiling baked in, but nobody has run it against a thousand-host fleet, and that's the honest answer.

Q. Why vanilla JS instead of React or another framework?

A. A deliberate choice to keep the frontend dependency-free — no build step, no framework version to maintain, everything served via Go's `embed.FS`. For a project this size it kept iteration fast; at real product scale a framework would likely earn its keep.

7 If something breaks

One fallback line per step, so a hiccup becomes a sentence instead of a stall.

STEP	IF IT DOESN'T COOPERATE
Ask Muster (self-hosted)	"The model's on CPU-only hardware, so this can take a while under load — while it finishes, let me show you [jump to entity map or reports] and circle back." Have a recent real answer's audit-log entry ready to show if it truly stalls.
Scanner import	Keep a pre-imported host on hand and open its vulnerability list directly — "here's what that import looks like once it's landed."
Live SIEM/backend switch	If the live switch doesn't show a new audit entry fast enough, open the audit log's existing history and point at the entry from the last time it was exercised.
Entity map rendering	If the browser is slow to lay it out, narrow the type filter to just CVEs and hosts first, then widen it once it's settled.
Network to the self-hosted model drops	Fall back to a screenshot of a previous real answer and be direct about it: "that's an actual answer from this model, captured earlier — the network between here and the model VM is the only thing that isn't live right now."

Go 1.24.7

Postgres 16

Vanilla JS/HTML/CSS

MIT License

Qwen3-30B-A3B (self-hosted)

CIS · HIPAA · NIST 800-53